顾名思义，就是把所有情况全都罗列出来，然后找出符合题目要求的那一个。因此，枚举是一种纯暴力的算法。

一般情况下，枚举策略都是会超时的。此时要先根据题目的数据范围来判断暴力枚举是否可以通过。如果不行的话，就要用后面要学的各种算法来进行优化（比如二分，双指针，前缀和与差分等）。

使用枚举策略时，重点思考枚举的对象（枚举什么），枚举的顺序（正序还是逆序），以及枚举的方式（普通枚举？递归枚举？二进制枚举）。

## 3.1 普通枚举

### 3.1.1 铺地毯

> **题目来源：** 洛谷
>
> **题目链接：** P1003 [NOIP2011 提高组] 铺地毯
>
> **难度系数：** ★

**【题目描述】**

> 为了准备一个独特的颁奖典礼，组织者在会场的一片矩形区域（可看做是平面直角坐标系的第一象限）铺上一些矩形地毯。一共有 $n$ 张地毯，编号从 $1$ 到 $n$。现在将这些地毯按照编号从小到大的顺序平行于坐标轴先后铺设，后铺的地毯覆盖在前面已经铺好的地毯之上。地毯铺设完成后，组织者想知道覆盖地面某个点的最上面的那张地毯的编号。
>
> 注意：在矩形地毯边界和四个顶点上的点也算被地毯覆盖。

**【输入描述】**

> 第一行，一个整数 $n$，表示总共有 $n$ 张地毯。
>
> 接下来的 $n$ 行中，第 $i+1$ 行表示编号 $i$ 的地毯的信息，包含四个正整数 $a,b,g,k$，每两个整数之间用一个空格隔开，分别表示铺设地毯的左下角的坐标 $(a,b)$ 以及地毯在 $x$ 轴和 $y$ 轴方向的长度。
>
> 第 $n+2$ 行包含两个正整数 $x$ 和 $y$，表示所求的地面的点的坐标 $(x,y)$。

**【输出描述】**

> 输出共 $1$ 行，一个整数，表示所求的地毯的编号；若此处没有被地毯覆盖则输出 $-1$。

**【示例一】**

> 输入：
>
> ```
> 3
> 1 0 2 3
> 0 2 3 3
> 2 1 3 3
> ```

> 2 2
>
> 输出：
>
> 3

**【解法】**

枚举所有的地毯，判断哪一个地毯能够覆盖 $(x, y)$ 这个位置。

优化枚举方式：

- 因为我们要的是最后一个能够覆盖 $(x, y)$ 位置的地毯，那么逆序枚举所有的地毯，第一次找到覆盖 $(x, y)$ 位置的就是结果；
- 如果从前往后枚举，我们至少要把所有地毯都枚举完，才能知道最终结果。

**【参考代码】**

代码块

```cpp
#include <iostream>

using namespace std;

const int N = 1e4 + 10;

int n;
int a[N], b[N], g[N], k[N];
int x, y;

int find()
{
    // 从后往前枚举
    for(int i = n; i >= 1; i--)
    {
        // 判断是否覆盖
        if(a[i] <= x && b[i] <= y && a[i] + g[i] >= x && b[i] + k[i] >= y)
        {
            return i;
        }
    }

    return -1;
}

int main()
{
    cin >> n;
```

```
29      for(int i = 1; i <= n; i++) cin >> a[i] >> b[i] >> g[i] >> k[i];
30      cin >> x >> y;
31
32      cout << find() << endl;
33
34      return 0;
35  }
```

### 3.1.2 回文日期

**题目来源：** 洛谷

**题目链接：** P2010 [NOIP2016 普及组] 回文日期

**难度系数：** ★

**【题目描述】**

在日常生活中，通过年、月、日这三个要素可以表示出一个唯一确定的日期。

牛牛习惯用 $8$ 位数字表示一个日期，其中，前 $4$ 位代表年份，接下来 $2$ 位代表月份，最后 $2$ 位代表日期。显然：一个日期只有一种表示方法，而两个不同的日期的表示方法不会相同。

牛牛认为，一个日期是回文的，当且仅当表示这个日期的 $8$ 位数字是回文的。现在，牛牛想知道：在他指定的两个日期之间包含这两个日期本身），有多少个真实存在的日期是回文的。

一个 $8$ 位数字是回文的，当且仅当对于所有的 $i(1 \leq i \leq 8)$ 从左向右数的第 $i$ 个数字和第 $9-i$ 个数字（即从右向左数的第 $i$ 个数字）是相同的。

例如：

- 对于 $2016$ 年 $11$ 月 $19$ 日，用 $8$ 位数字 $20161119$ 表示，它不是回文的。
- 对于 $2010$ 年 $1$ 月 $2$ 日，用 $8$ 位数字 $20100102$ 表示，它是回文的。
- 对于 $2010$ 年 $10$ 月 $2$ 日，用 $8$ 位数字 $20101002$ 表示，它不是回文的。

**【输入描述】**

输入包括两行，每行包括一个 $8$ 位数字。

第一行表示牛牛指定的起始日期。

第二行表示牛牛指定的终止日期。

保证 $date_i$ 和都是真实存在的日期，且年份部分一定为 $4$ 位数字，且首位数字不为 $0$ 。

保证 $date_1$ 一定不晚于 $date_2$ 。

**【输出描述】**

输出一行，包含一个整数，表示在 $date_1$ 和 $date_2$ 之间，有多少个日期是回文的。

**【示例一】**

> 输入：
>
> 20110101
>
> 20111231
>
> 输出：
>
> 1

**【示例二】**

> 输入：
>
> 20000101
>
> 20101231
>
> 输出：
>
> 2

**【解法】**

枚举所有日月的组合，然后根据回文的特性推出年份，然后比较这个数字时候在题目给定的区间内。

**【参考代码】**

代码块

```cpp
// 策略三：枚举所有的月日的组合

#include <iostream>

using namespace std;

int x, y;
int day[] = {0, 31, 29, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31};

int main()
{
    cin >> x >> y;

    int ret = 0;

    // 枚举月日的组合
    for(int i = 1; i <= 12; i++)
    {
        for(int j = 1; j <= day[i]; j++)
        {
```

```
21                int k = j % 10 * 1000 + j / 10 * 100 + i % 10 * 10 + i / 10;
22                int num = k * 10000 + i * 100 + j;
23                if(x <= num && num <= y) ret++;
24            }
25        }
26
27        cout << ret << endl;
28
29        return 0;
30    }
```

### 3.1.3 扫雷

**题目来源：** 洛谷

**题目链接：** P2327 [SCOI2005] 扫雷

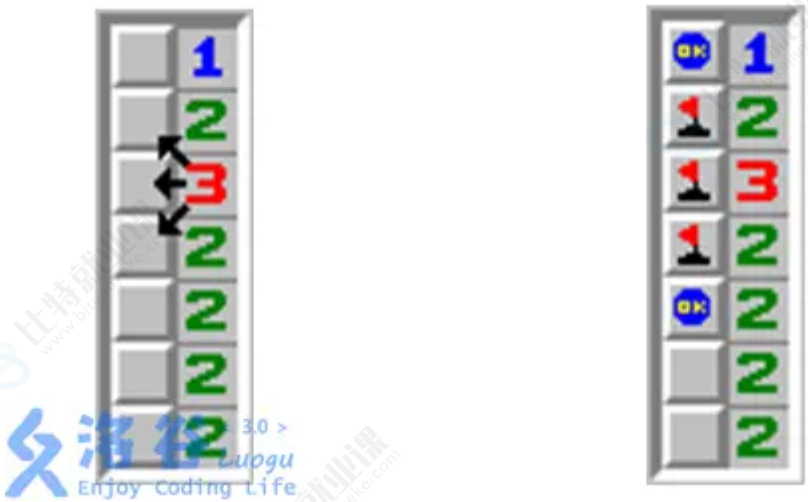
**难度系数：** ★★

**【题目描述】**

相信大家都玩过扫雷的游戏。那是在一个 $n \times m$ 的矩阵里面有一些雷，要你根据一些信息找出雷来。

万圣节到了，“余” 人国流行起了一种简单的扫雷游戏，这个游戏规则和扫雷一样，如果某个格子没有雷，那么它里面的数字表示和它 $8$ 连通的格子里面雷的数目。

现在棋盘是 $n \times 2$ 的，第一列里面某些格子是雷，而第二列没有雷，如下图：




由于第一列的雷可能有多种方案满足第二列的数的限制，你的任务即根据第二列的信息确定第一列雷有多少种摆放方案。

**【输入描述】**

第一行为 $N$，第二行有 $N$ 个数，依次为第二列的格子中的数。 $(1 \leq N \leq 10000)$

**【输出描述】**

> 一个数，即第一列中雷的摆放方案数。

**【示例一】**

> 输入：
>
> 2
>
> 1 1
>
> 输出：
>
> 2

**【解法】**

我们发现，当第一列中，第一行的小格子的状态确定了之后，其实后续行的状态也跟着固定下来。而第一列中，第一行的状态要么有雷，要么没有雷，所以最终的答案就在 $0, 1, 2$ 中。

因此，我们枚举第一列中，第一行的两种状态：要么有雷，要么没雷。然后依次计算剩下行的值，看看是否能满足所给的数据。

**【参考代码】**

代码块

```cpp
#include <iostream>

using namespace std;

const int N = 1e4 + 10;

int n;
int a[N], b[N];

// 不放地雷
int check1()
{
    a[1] = 0;
    for(int i = 2; i <= n + 1; i++)
    {
        a[i] = b[i - 1] - a[i - 1] - a[i - 2];
        if(a[i] < 0 || a[i] > 1) return 0;
    }
    if(a[n + 1] == 0) return 1;
    else return 0;
}

```

```cpp
// 放地雷
int check2()
{
    a[1] = 1;
    for(int i = 2; i <= n + 1; i++)
    {
        a[i] = b[i - 1] - a[i - 1] - a[i - 2];
        if(a[i] < 0 || a[i] > 1) return 0;
    }
    if(a[n + 1] == 0) return 1;
    else return 0;
}

int main()
{
    cin >> n;
    for(int i = 1; i <= n; i++) cin >> b[i];

    int ret = 0;
    ret += check1(); // a[1] 不放地雷
    ret += check2(); // a[1] 放地雷

    cout << ret << endl;

    return 0;
}
```

## 3.2 二进制枚举

二进制枚举：用一个数二进制表示中的 $0/1$ 表示两种状态，从而达到枚举各种情况。

- 利用二进制枚举时，会用到一些**位运算**的知识。不熟悉同学需要去补一下**位运算相关的知识**。
- 关于用二进制中的 $0/1$ 表示状态这种方法，会在动态规划章节中的状态压缩 $dp$ 中继续使用到。
- 二进制枚举的方式也可以用递归实现，后续递归课程中会再讲到。

### 3.2.1 子集

> **题目来源：** 力扣
>
> **题目链接：** 子集
>
> **难度系数：** ★

**【题目描述】**

> 给你一个整数数组 $nums$ ，数组中的元素互不相同。返回该数组所有可能的子集（幂集）。
>
> 解集不能包含重复的子集。你可以按任意顺序返回解集。

**【示例一】**

> 输入：
>
> nums = [1,2,3]
>
> 输出：
>
> [[],[1],[2],[1,2],[3],[1,3],[2,3],[1,2,3]]

**【ACM 模式与核心代码模式】**

1. **ACM 模式**

ACM 模式一般是竞赛和笔试面试常用的模式，就是只给你一个题目描述，外加输入样例和输出样例，不会给你任何的代码。此时，选手或者应聘者需要根据题目要求，自己完成如下任务：

> 1. 头文件的包含
> 2. main 函数的设计
> 3. 自己定义程序所需的变量和容器（数组、哈希表等等）
> 4. 数据的输入（根据题目叙述控制输入数据的格式）
> 5. 数据的处理（各种函数接口的设计）
> 6. 数据的输出（根据题目叙述控制返回数据的格式）

总而言之，ACM 模式相当于给你题目和一个空白的代码框，让你自己设计程序来解决问题。

因此，ACM 模式更加能够锻炼代码能力，以及处理问题的整体逻辑。

2. **核心代码模式**

相比较于 ACM 模式，核心代码模式就只用实现主要功能。

> 1. 核心代码模式不需要你处理头文件、输入和输出等乱七八糟的东西，只是甩给你一个函数接口。你的任务就仅仅是完成这个函数；
> 2. 在这一个函数接口中，函数头部分会传给你需要的数据，直接使用即可；
> 3. 在你完成这个函数并且提交之后，后台会调用你所写的函数，并且根据你返回的结果测试是否正确。

这种情况下，我们只需完成核心的函数接口，无需考虑数据的输入和输出。

例如：leetcode 上一道简单的核心代码模式的题：2235. 两整数相加

核心代码模式代码：

代码块

```
// 题目已经把接口给你设计好了，并且只给你一个函数接口
// 你只需要实现算法的主要逻辑，然后返回即可
int sum(int num1, int num2)
{
    return num1 + num2; // 只需完成算法的核心逻辑，不用考虑数据的输入和输出
}
```

因此，核心代码模式主要是锻炼算法的逻辑，无需考虑繁琐重复的输入和输出，重点在核心算法的实现上。

我们的课程中会出现少量的核心代码模式的题目，因为这类题目相对简单，非常适合作为学习阶段来练习。

**【解法】**

枚举 $1$ ~ $1 << n$ 之间所有的数，每一个数的二进制中 $1$ 的位置可以表示数组中对应位置选上该元素。那么 $1$ ~ $1 << n$ 就可以枚举出原数组中所有的子集。

根据枚举的每一个状态，选出原数组中对应的元素，然后存在结果数组中。

**【参考代码】**

代码块

```
class Solution
{
public:
    vector<vector<int>> subsets(vector<int>& nums)
    {
        vector<vector<int>> ret;
        int n = nums.size();

        // 枚举所有的状态
        for(int st = 0; st < (1 << n); st++)
        {
            // 根据 st 的状态，还原出要选的数
            vector<int> tmp; // 从当前选的子集
            for(int i = 0; i < n; i++)
            {
                if((st >> i) & 1) tmp.push_back(nums[i]);
```

```
            }
            ret.push_back(tmp);
        }

        return ret;
    }
};
```

### 3.2.2 费解的开关

**题目来源：** 洛谷

**题目链接：** P10449 费解的开关

**难度系数：** ★★★

**【题目描述】**

你玩过 “拉灯” 游戏吗？ 25 盏灯排成一个 $5 \times 5$ 的方形。每一个灯都有一个开关，游戏者可以改变它的状态。

每一步，游戏者可以改变某一个灯的状态。游戏者改变一个灯的状态会产生连锁反应：和这个灯上下左右相邻的灯也要相应地改变其状态。

我们用数字 “1” 表示一盏开着的灯，用数字 “0” 表示关着的灯。下面这种状态

10111

01101

10111

10000

11011

在改变了最左上角的灯的状态后将变成：

01111

11101

10111

10000

11011

再改变它正中间的灯后状态将变成：

01111

11001

11001

10100

11011

给定一些游戏的初始状态，编写程序判断游戏者是否可能在 $6$ 步以内使所有的灯都变亮。

**【输入描述】**

第一行有一个正整数 $n$ ，代表数据中共有 $n$ 个待解决的游戏初始状态。
以下若干行数据分为 $n$ 组，每组数据有 $5$ 行，每行 $5$ 个字符。每组数据描述了一个游戏的初始状态。各组数据间用一个空行分隔。
对于 $30\%$ 的数据， $n \leq 5$ ;
对于 $100\%$ 的数据， $n \leq 500$ 。

**【输出描述】**

输出数据一共有 $n$ 行，每行有一个小于等于 $6$ 的整数，它表示对于输入数据中对应的游戏状态最少需要几步才能使所有灯变亮。
对于某一个游戏初始状态，若 $6$ 步以内无法使所有灯变亮，请输出 “-1”。

**【示例一】**

输入：

3

00111

01011

10001

11010

11100

11101

11101

11110

11111

11111

01111

11111

11111

```
11111
11111
```

输出：

```
3
2
-1
```

**【解法】**

在这个「拉灯游戏」中我们可以得到「三个性质」：

1. 每一个开关「最多只会被按一次」。因为按两次及以上是没有意义的，只会让按的次数增多；
2. 按每一个开关的「先后顺序」不会影响最后的结果。可以想象，当所有开关按的方式确定之后，每一个开关被改变的「次数」也就被确定了，也就是说不管你先按谁后按谁，改变的次数是固定的，那么结果就是固定的；
3. 如果「确定了第一行」的按法，后续行的按法就也固定下来了（这里可以参考之前《扫雷》这道题，有相似点）。因为第一行的按法固定之后，第二行的按法需要把第一行「全部点亮」；当第二行的按法确定之后，第三行的按法需要把第二行「全部点亮」.....，依次类推，后续行的按法就都确定下来了。

有了这三个性质，那么我们的核心思路就是：

1. 暴力「枚举」第一行的所有按法；
2. 然后根据第一行的按法，计算出当前行以及下一行被按之后的结果；
3. 根据上一行被按了之后的状态，确定当前行的按法，然后重复 $2$ 操作；
4. 最后判断最后一行是否全部都亮。

接下来考虑每一步如何「优美」的实现。为了方便起见，我们读取原数据的时候把所有的 $1$ 当成 $0$ ，把所有的 $0$ 当成 $1$ ，这样题目要求的全亮，就变成全灭，后续各种操作都非常舒服。具体原因大家可以在学会全灭的解法之后，尝试全亮的写法，你会发现好多地方都让人很难受~

1. 读取数据时，我们直接用「二进制」存每一行的状态：

   比如： $00101$ ，对应的就是 $(00101)_2 = 5$ 。这样我们就可以用「位运算」快速实现一些操作，方便之处会在后续算法原理中体现；

2. 枚举第一行所有的按法：

   枚举 $1 \sim (1 << 5) - 1$ 之间所有的数，如果二进制表示中第 $i$ 位是 $1$ 就表示第一行的第 $i$ 位被按；

3. 如何计算某个状态下，一共按了多少次：

相当于计算二进制表示中 1 的个数，常规操作，无需多言；

4. 如何优美的根据当前行的按法 $push$，得到当前行 $a[i]$ 以及下一行 $a[i+1]$ 被按 $push$ 了之后的状态：

   a. 当前行：被按的位置会影响「当前位置」以及「左右两个位置」的状态，如果状态是 0 会被变成 1，如果状态是 1 会被变成 0，不正好是 $x \hat{} 1$ 之后的结果么？又因为会改变「当前位置」以及「左右两个位置」，所以 $a[i]$ 的最终状态就是：
   $a[i] = a[i] \hat{} push \hat{} (push >> 1) \hat{} (push << 1)$；

   其中，$push << 1$ 有可能会让第 5 位变成 1，这一位是一个「非法」的位置，有可能影响后续判断，我们要「截断高位」：$(push << 1) \hat{} ((1 << 5) - 1)$；

   最终：$a[i] = a[i] \hat{} push \hat{} (push >> 1) \hat{} ((push << 1) \hat{} ((1 << 5) - 1))$；

   b. 下一行：当前行的 $push$ 只会对下一行「对应的位置」做修改：$a[i+1] = a[i+1] \hat{} push$；

   发现没，使用「二进制」表示存状态之后，改变的时候只用使用「位运算」即可，不然还要写 $for$ 循环来改变每一个位置的值，是不是很优雅~

5. 求出当前行被按了之后的结果，如何求出下一行的按法：

   巨简单，当前行怎么亮，下一行就怎么按，这样就可以把当前行亮的位置暗灭：$nextpush = a[i]$，注意此时的 $a[i]$ 是被按了之后的状态。

6. 判断最后一行是否全灭：

   判断 $a[4] == 0$ 即可，我们开头「反着存储」的优势就体现出来了。

**这类题非常锻炼代码能力，希望大家在看完详细的算法原理之后，能够自己先尝试实现这个算法。**

**【参考代码】**

代码块

```cpp
#include <iostream>
#include <cstring>

using namespace std;

const int N = 10;

int n = 5;
int a[N]; // 用二进制表示，来存储灯的状态
int t[N]; // 备份 a 数组

// 计算 x 的二进制表示中一共有多少个 1
int calc(int x)
{
    int cnt = 0;
```

```
16      while(x)
17      {
18          cnt++;
19          x &= x - 1;
20      }
21      return cnt;
22  }
23
24  int main()
25  {
26      int T; cin >> T;
27
28      while(T--)
29      {
30          // 多组测试时，一定要注意清空之前的数据
31          memset(a, 0, sizeof a);
32
33          for(int i = 0; i < n; i++)
34          {
35              for(int j = 0; j < n; j++)
36              {
37                  char ch; cin >> ch;
38                  // 存成相反的
39                  if(ch == '0') a[i] |= 1 << j;
40              }
41          }
42
43          int ret = 0x3f3f3f3f; // 统计所有合法的按法中的最小值
44          // 枚举第一行所有的按法
45          for(int st = 0; st < (1 << n); st++)
46          {
47              memcpy(t, a, sizeof a);
48              int push = st; // 当前行的按法
49              int cnt = 0; // 统计当前按法下一共按了多少次
50
51              // 依次计算后续行的结果以及按法
52              for(int i = 0; i < n; i++)
53              {
54                  cnt += calc(push);
55                  // 修改当前行被按的结果
56                  t[i] = t[i] ^ push ^ (push << 1) ^ (push >> 1);
57                  t[i] &= (1 << n) - 1; // 清空影响
58                  // 修改下一行的状态
59                  t[i + 1] ^= push;
60                  // 下一行的按法
61                  push = t[i];
62              }
```

```
63
64              if(t[n - 1] == 0) ret = min(ret, cnt);
65          }
66
67          if(ret > 6) cout << -1 << endl;
68          else cout << ret << endl;
69      }
70
71
72      return 0;
73  }
```

### 3.2.3 Even Parity

**题目来源：** 洛谷

**题目链接：** Even Parity

**难度系数：** ★★★

**【题目描述】**

给你一个 $n \times n$ 的 $01$ 矩阵（每个元素非 $0$ 即 $1$），你的任务是把尽量少的 $0$ 变成 $1$，使得原矩阵变为偶数矩阵（矩阵中每个元素的上、下、左、右的元素（如果存在的话）之和均为偶数）。

**【输入描述】**

输入的第一行为数据组数 $T(T \leq 30)$。每组数据：第一行为正整数 $n(1 \leq n \leq 15)$；接下来的 $n$ 行每行包含 $n$ 个非 $0$ 即 $1$ 的整数，相邻整数间用一个空格隔开。

**【输出描述】**

对于每组数据，输出被改变的元素的最小个数。如果无解，输出 $-1$。

**【示例一】**

输入：

```
3
3
0 0 0
0 0 0
0 0 0
3
0 0 0
```

1 0 0

0 0 0

3

1 1 1

1 1 1

0 0 0

输出：

Case 1: 0

Case 2: 3

Case 3: -1

**【解法】**

我们发现：

1. 每一个 $0$ 如果变成 $1$ ，只会「变一次」；
2. 当第一行的「最终状态」确定之后，第二行的「最终状态」也会确定

所以我们可以「暴力枚举」第一行的最终状态，在这个最终状态「合法」的前提下，「递推」出来第二行的状态，「以此类推」下去。

考虑一下几个问题：

1. 如何枚举第一行所有的「最终状态」 $st$ ：

   枚举 $1 \sim (1 << n) - 1$ 之间所有的数，「每一个数」就是第一行的最终状态；

2. 由于本题只能 $0$ 变 $1$ ，所以我们还要「判断」每一行的最终状态 $y$ 「是否合法」：

   很简单，比较初始状态 $x$ 以及最终状态 $y$ 中「二进制表示的每一位」，如果是 $0$ 变 $1$ ，就是「合法」操作，计数；如果是 $1$ 变 $0$ ，「非法」操作，直接「跳出本次循环」，枚举第一行的下一个状态；

3. 当前行的最终状态 $a[i]$ 确定之后，如何「递推」下一行的最终状态 $a[i+1]$ ：

   规则是当前位置「上下左右」 $1$ 的个数之和是「偶数」，根据「异或」运算「无进位相加」的特性，正好就是上下左右位置「异或」的结果是 $0$ 。那么下一行对应位置的状态就是「当前行右移一位」与「当前行左移一位」与「上一行对应位置」异或的结果：
   $a[i+1] = (a[i] >> 1) \hat{} (a[i] << 1) \hat{} a[i-1]$ ;

   其中 $a[i] << 1$ 会造成不合法的位置是 $1$ 的情况，注意「高位截断」：
   $(a[i] << 1) \ \& \ ((1 << n) - 1)$

**【参考代码】**

代码块

```cpp
#include <iostream>
#include <cstring>

using namespace std;

const int N = 20;

int n;
int a[N]; // 用二进制存储状态
int t[N]; // 备份

// 判断 x->y 是否合法
// 返回 -1，表示不合法
// 其余的数，表示合法，并且表示 0->1 的次数
int calc(int x, int y)
{
    int sum = 0;

    for(int i = 0; i < n; i++)
    {
        if(((x >> i) & 1) == 0 && ((y >> i) & 1) == 1) sum++;
        if(((x >> i) & 1) == 1 && ((y >> i) & 1) == 0) return -1;
    }

    return sum;
}

int solve()
{
    int ret = 0x3f3f3f3f; // 记录最小的改变次数

    // 枚举第一行的最终状态
    for(int st = 0; st < (1 << n); st++)
    {
        memcpy(t, a, sizeof a);

        int change = st;
        int cnt = 0; // 统计 0->1 的次数
        bool flag = 1;

        for(int i = 1; i <= n; i++)
        {
            // 先判断 change 是否合法
```

```
            int c = calc(t[i], change);
            if(c == -1)
            {
                flag = 0;
                break;
            }

            cnt += c; // 累加次数
            // 当前行的最终状态
            t[i] = change;
            // 计算下一行的最终状态
            change = t[i - 1] ^ (t[i] << 1) ^ (t[i] >> 1);
            change &= (1 << n) - 1;
        }
        if(flag) ret = min(ret, cnt);
    }

    if(ret == 0x3f3f3f3f) return -1;
    else return ret;
}

int main()
{
    int T; cin >> T;

    for(int k = 1; k <= T; k++)
    {
        // 多组测试数据，记得清空
        memset(a, 0, sizeof a);

        cin >> n;
        for(int i = 1; i <= n; i++) // 避免越界访问
        {
            for(int j = 0; j < n; j++)
            {
                int x; cin >> x;
                if(x) a[i] |= 1 << j;
            }
        }

        printf("Case %d: %d\n", k, solve());
    }

    return 0;
}
```